Informe de Actividades — TP N.º 1

Materia: Base de Datos I — Tecnicatura Universitaria en Programación (UTN) Proyecto integrador: Food Store (sistema de gestión de pedidos) Motor: PostgreSQL 17+ · Cliente: DBeaver / psql Integrantes: Guillot Tiago, Monardez Maximo, Ruiz Matias, Mendez Joaquin, Oviedo Thiago y Chacon Facundo Repositorio: github.com/thiaguchi-stack/DB-TrabajosPracticos-Com4

Este informe recorre cada actividad de la consigna del TP1 y muestra qué se produjo, dónde está el entregable y cómo se verificó. El desarrollo completo, con el razonamiento extenso y las respuestas a todas las preguntas guía, está en `TP1_FoodStore_Desarrollo.md`.

Resumen de cumplimiento

Actividad	Peso	Entregable	Estado
Parte 1 — Modelo entidad-relación	25%	`diagrama/diagrama_er.dbml`, `diagrama/TP1_Diagrama_ER.pdf`, diccionario en el desarrollo	Completa
Parte 2 — Derivación al modelo relacional	20%	Tablas con PK/FK y justificación en el desarrollo	Completa
Parte 3 — Normalización 1FN → 3FN/BCNF	25%	Siete pasos, dependencias funcionales y tablas finales	Completa
Parte 4 — DDL en PostgreSQL	25%	`sql/schema.sql` ejecutable	Completa
Uso de IA	—	`DUIA.md`	Completa

Actividad 1 — Modelo entidad-relación

Consigna. Construir el diagrama ER completo del dominio Food Store, con entidades, atributos, claves primarias y candidatas, cardinalidades y participación total o parcial justificada por las reglas R1-R7. Notación elegida: Pata de Gallo (Crow's Foot), sostenida en todo el diagrama.

Desarrollo. Se identificaron cinco entidades: `categoria`, `producto`, `cliente`, `pedido` y `linea_pedido`. Las relaciones son:

Relación	Cardinalidad	Participación	Regla
categoria–producto	1:N	producto total, categoria parcial	R1: todo producto tiene exactamente una categoría; una categoría puede no tener productos
cliente–pedido	1:N	pedido total, cliente parcial	R2: todo pedido tiene un cliente; un cliente puede no haber pedido nada
pedido–producto	N:M	pedido total respecto de sus líneas, producto parcial	R3/R4: un pedido incluye varios productos y un producto aparece en muchos pedidos

Claves y atributos. `categoria.id`, `producto.id`, `cliente.id` y `pedido.id` son claves primarias sustitutas. `cliente.email` es clave candidata alternativa (R6) y `(categoria_id, nombre)` identifica un producto dentro de su categoría. La tabla `linea_pedido` tiene clave primaria compuesta `(pedido_id, producto_id)` y los atributos propios de la relación: `cantidad` y `precio_unitario`.

Respuestas guía (síntesis). La relación producto–pedido no puede ser 1:N porque se perderían líneas del pedido (el pedido 1 tiene Muzzarella y Coca) o la reutilización del producto en varios pedidos; además, `cantidad` y `precio_unitario` son atributos de la línea. En categoria–producto, `producto` tiene participación total y `categoria` parcial: invertirlo permitiría productos sin categoría y prohibiría categorías vacías. El nombre del cliente se separó en `nombre` y `apellido`; el nombre del producto se dejó atómico porque el negocio no necesita sus partes.

Entregable y verificación. El diagrama está en `diagrama/diagrama_er.dbml` (para dbdiagram.io) y exportado a `diagrama/TP1_Diagrama_ER.pdf`. El diccionario completo de entidades, atributos, tipos conceptuales y restricciones está en el desarrollo.

Actividad 2 — Derivación al modelo relacional

Consigna. Convertir el ER en tablas, resolver las relaciones 1:N con clave foránea en el lado N y la relación N:M con una tabla intermedia, y justificar la clave primaria elegida para esa tabla.

Desarrollo. Las tablas derivadas son:

```
categoria     (id, nombre, activo, created_at)

producto      (id, nombre, descripcion, precio, stock, activo,
               categoria_id -> categoria, created_at)

cliente       (id, nombre, apellido, email, telefono, activo, created_at)

pedido        (id, fecha, forma_pago, cliente_id -> cliente, created_at)

linea_pedido  (pedido_id -> pedido, producto_id -> producto,
               cantidad, precio_unitario)
```

`producto.categoria_id` y `pedido.cliente_id` son FK `NOT NULL` porque la participación de producto y pedido es total (R1 y R2). La tabla intermedia conserva `cantidad` y `precio_unitario` como atributos propios (R4).

Justificación de la PK. Se eligió la clave compuesta `(pedido_id, producto_id)`: la regla de negocio prohíbe repetir un producto dentro del mismo pedido, así que la combinación identifica cada línea y hace cumplir la regla directamente en el motor. Una clave sustituta obligaría a agregar igualmente un `UNIQUE (pedido_id, producto_id)`.

Respuestas guía (síntesis). Si las FK de la tabla intermedia fueran nulables podrían existir líneas huérfanas, totales por pedido incorrectos y duplicados que la PK compuesta no detecta porque `NULL <> NULL`. Si un pedido pudiera existir sin líneas, la participación de pedido en la relación pasaría de total a parcial y el esquema no podría garantizar por declaración que todo pedido tenga al menos una línea; se necesitarían triggers o lógica de aplicación.

Actividad 3 — Normalización hasta 3FN/BCNF

Consigna. Partir de la planilla plana de ventas, determinar la clave, listar las dependencias funcionales y desarrollar paso a paso 1FN, 2FN, 3FN y BCNF, incluyendo las preguntas de integración.

Paso 1 — Clave. La relación universal es `venta (nro_pedido, fecha, cliente, producto, categoria, precio_unitario, cantidad, subtotal, forma_pago)`. La única clave candidata es `(nro_pedido, producto)`: `nro_pedido` solo no alcanza porque el pedido 1 tiene dos líneas, y `producto` solo no alcanza porque la Muzzarella aparece en los pedidos 1, 3 y 5.

Paso 2 — Dependencias funcionales.

Dependencia	Justificación
`nro_pedido → fecha, cliente, forma_pago`	Cada pedido tiene una fecha, un cliente y una forma de pago únicos
`producto → categoria`	Cada producto pertenece a una única categoría
`(nro_pedido, producto) → cantidad`	La cantidad es de la línea
`(nro_pedido, producto) → precio_unitario`	El precio depende del producto en ese pedido: la Muzzarella vale 1000 en el pedido 1 y 1050 en el pedido 3
`cantidad, precio_unitario → subtotal`	El subtotal es derivado (`cantidad × precio_unitario`)

Paso 3 — 1FN. Todos los atributos son atómicos: no hay listas ni grupos repetidos; los productos de un pedido ocupan filas distintas. Cumple 1FN.

Paso 4 — 2FN. `fecha`, `cliente` y `forma_pago` dependen solo de `nro_pedido`; `categoria` depende solo de `producto`. Son dependencias parciales. Se separan `pedido (nro_pedido, fecha, cliente, forma_pago)` y `linea (nro_pedido, producto, precio_unitario, cantidad, subtotal)`.

Paso 5 — 3FN. En `linea`, `(nro_pedido, producto) → producto → categoria` es transitiva: se extrae `producto (producto, categoria)`. También `cantidad, precio_unitario → subtotal` es una dependencia entre atributos no clave: `subtotal` se elimina por ser derivado.

Paso 6 — BCNF. En `pedido` el único determinante es `nro_pedido` (PK); en `producto`, `producto` (PK); en `linea_pedido`, `(nro_pedido, producto)` (PK). No existe `producto → precio_unitario`, y los datos lo demuestran. Todas las tablas están en BCNF.

Paso 7 — Tablas finales.

```
pedido       (nro_pedido, fecha, cliente, forma_pago)
producto     (producto, categoria)
linea_pedido (nro_pedido, producto, precio_unitario, cantidad)
```

Preguntas de integración (síntesis). La normalización recupera la estructura implícita en los datos históricos y coincide con el ER, pero el ER aporta atributos que la planilla no tenía (`email`, `telefono`, `stock`, `descripcion`, `activo`, IDs y fechas de creación). Modelar desde el enunciado captura el dominio completo; normalizar desde datos valida el diseño. `subtotal` no conviene almacenarlo: es una multiplicación trivial, introduce dependencia entre atributos no clave (violaría 3FN) y el precio congelado ya preserva la trazabilidad histórica.

Actividad 4 — DDL en PostgreSQL

Consigna. Escribir `schema.sql` con tipos adecuados, PK y FK explícitas, `UNIQUE`, al menos tres `CHECK`, `NOT NULL` coherente con la participación, `DEFAULT` e índices justificados.

Desarrollo. `sql/schema.sql` crea el tipo `forma_pago AS ENUM ('EFECTIVO', 'TARJETA', 'TRANSFERENCIA')` y las cinco tablas. Decisiones principales:

Elemento	Implementación	Motivo
Identificadores	`BIGINT GENERATED ALWAYS AS IDENTITY`	Clave autogenerada, sin manejo manual
Montos	`NUMERIC(10,2)`	Precisión fija, nunca punto flotante
Fechas	`TIMESTAMPTZ NOT NULL DEFAULT now()`	Zona horaria y valor automático
Baja lógica	`activo BOOLEAN NOT NULL DEFAULT TRUE`	R7: no se eliminan productos, categorías ni clientes
Unicidad	`cliente.email UNIQUE`, `categoria.nombre UNIQUE`, `UNIQUE (categoria_id, nombre)`	Claves candidatas del ER
CHECK	`precio >= 0`, `stock >= 0`, `cantidad > 0`, `precio_unitario >= 0`	R5 y coherencia de la venta
FK de pedido	`cliente_id REFERENCES cliente(id) ON DELETE RESTRICT`	Conservar historial de ventas
FK de línea	`pedido_id ... ON DELETE CASCADE`, `producto_id ... ON DELETE RESTRICT`	Las líneas siguen al pedido; el producto facturado no se borra
Índices	`pedido(cliente_id)`, `producto(categoria_id)`, `pedido(fecha)`	Aceleran pedidos por cliente, carta por categoría y ventas por rango de fechas

Verificación. El script se ejecutó completo sobre una base vacía en PostgreSQL 18.6 (compatible con 17) sin errores. Se verificaron las tablas, el tipo enumerado, las restricciones y los índices con `\dt` y `\d+`.

Actividad 5 — Uso de IA y puesta a punto del flujo

Consigna. Documentar en la DUIA la herramienta usada, para qué y con qué prompts o specs, dejando registrada la puesta a punto del flujo de IA y el primer ejercicio de integridad versionado en Git.

Desarrollo. `DUIA.md` declara el uso de OpenCode como motor primario para el análisis del enunciado, el diagrama, la derivación, la normalización y el DDL, con verificación humana línea por línea y ejecución en DBeaver/PostgreSQL. El flujo de trabajo incluye repositorio Git, copia de la base, transacción y respaldo; esas prácticas se profundizan en el TP2, donde se agregan el protocolo de seguridad, los triggers de integridad, el laboratorio de concurrencia y la lectura crítica de scripts generados por IA.

Verificación. Cada script fue leído antes de ejecutarse. `schema.sql` se probó sobre una base vacía y las decisiones de diseño (PK compuesta, `ON DELETE`, no almacenar `subtotal`) fueron entendidas y defendidas por el alumno.

Checklist final

- [x] Diagrama ER con entidades, atributos, claves, cardinalidades y participación.
- [x] Diccionario de datos y respuestas a las preguntas guía.
- [x] Derivación completa al modelo relacional con PK y FK.
- [x] Normalización desarrollada paso a paso hasta BCNF.
- [x] `schema.sql` ejecutable, con restricciones e índices justificados.
- [x] DUIA completa y verificación humana registrada.